# San José State University
# Department of Computer Engineering
# CMPE 146-01, Real-Time Embedded System Co-Design, Fall 2025

# Lab Assignment 1

**Due date:** 09/07/2025, Sunday

## 1. Overview

In this assignment, we will be familiarized with the Texas Instruments MSPM0G3507 LaunchPad Development Kit and the TI IDE (Integrated Development Environment). The development kit features the MSPM0G3507, an 80-MHz ARM Cortex-M0+ microcontroller (MCU). We will set up the IDE and the SDK (Software Development Kit) for the microcontroller. We will also create simple programs to familiarize the application development process.

## 2. Resources

LaunchPad Development Kit website: https://www.ti.com/tool/LP-MSPM0G3507#tech-docs

MSPM0G3507 MCU website: https://www.ti.com/product/MSPM0G3507

Key documentation of the hardware and software:

LaunchPad Development Kit User's Guide: PDF, HTML

MCU Datasheet: PDF, HTML

MCU Technical Reference: PDF

MSPM0 Driver Library (DriverLib): HTML

## 3. Exercise 1 CCS and SDK

In this exercise, we will set up the TI IDE tool, CCS (Code Composer Studio), and the SDK targeted for the specific MCU used on the LaunchPad.

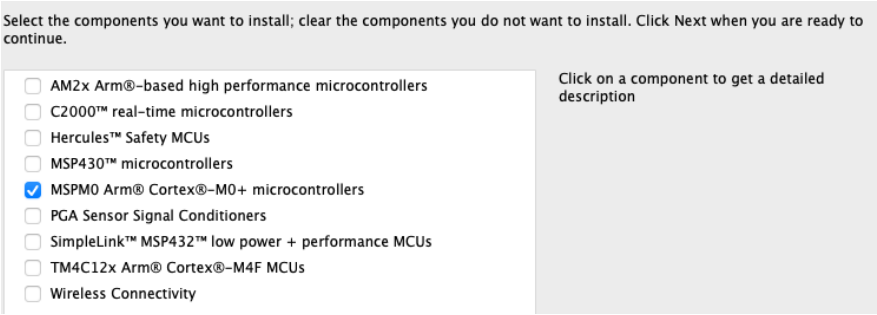
Code Composer Studio User's Guide can be found at https://software-dl.ti.com/ccs/esd/documents/users_guide/index.html. We will describe a brief procedure of the installation. Details can be found in Chapter 3 of the user's guide, https://software-dl.ti.com/ccs/esd/documents/users_guide/ccs_installation.html.

Go to the "Code Composer Studio Downloads" website: https://www.ti.com/tool/download/CCSTUDIO. We are going to download the latest version, 20.2.0. Download the "single file (offline)" installer. CCS is supported on three different OS platforms: Windows, macOS and Linux.

If you use a Windows machine, for example, click on the "Windows single file (offline) installer for Code Composer Studio IDE (all features, devices)" link. You will download the *CCS_20.2.0.00012_win64.zip* file to your local disk. The file size is about 1 GB, so it may take a few minutes to complete the download. After the download is complete, unzip the file. Run the setup program *ccs_setup_20.2.0.00012.exe*.

If you use a macOS machine, click on the "macOS single file (offline) installer for Code Composer Studio IDE (all features, devices)" link. You will download the *CCS_20.2.0.00012_mac_x86.dmg*. Open the disk image file. Run the setup program *ccs_setup_20.2.0.00012.app*.

Follow the on-screen instructions to install the CCS. When you get to the "Select Components" screen, only select "MSPM0 Arm Cortex-M0+ microcontrollers." It is a large installer, so the entire installation process may take more than half an hour on a slow machine.




After finishing the CCS installation, do not start CCS yet. We still need to install the SDK.

To download the SDK, go to https://www.ti.com/tool/MSPM0-SDK#downloads. After selecting the installer for your platform (see below), you will need to log in to your TI account (create one if you don't have it) and answer a few questions before you can download the file.

If you use a Windows machine, download the SDK installer *mspm0_sdk_2_05_01_00.exe*. Execute the program to install the SDK in your local disk. If the OS does not allow you to run the program due to some security reason, open a Command Prompt window (run the cmd program) with admin privilege, go to the folder (with the cd command) where the executable file is, and type the filename to run the program. When it is done, you should see a new folder created in the TI software folder, for example, *C:\ti\simplelink_msp432p4_sdk_3_40_01_02*.

If you use a macOS machine, download the SDK installer *mspm0_sdk_2_05_01_00.app.zip*. Open the file to extract the setup program *mspm0_sdk_2_05_01_00.app*. Run the program. A new folder will be created in the TI software folder, for example, */Applications/ti/mspm0_sdk_2_05_01_00*.

**Lab Report Submission**

1. List any issues that you may have encountered during the setups. Describe how they were resolved.
2. Include a screenshot of the TI folder showing the CCS and SDK folders.

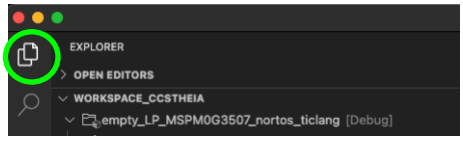
## 4. Exercise 2 Hello world

In this exercise, we will create a new project on CCS to have the LaunchPad send a simple message to the CCS Console I/O (CIO).

Connect the LaunchPad to your laptop/desktop with the USB cable (that comes with the LaunchPad). Then start CCS.

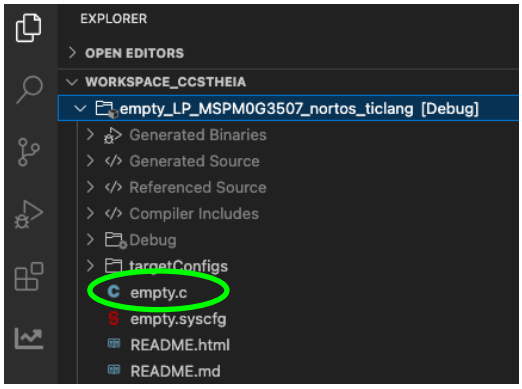
Quite often, it is much quicker to import an example empty project and modify it to suit our needs than to create a new project from scratch. On the CCS menu bar, click on <File>< Import Project(s)…>. On the pop-up "Import Projects" dialog box, click on the "Browse…" button. Go to the example project folder, for example, on Windows, it would be something like *C:\ti\mspm0_sdk_2_05_01_00\examples\nortos\LP_MSPM0G3507\driverlib\empty*. On macOS, it would be */Applications/ti/mspm0_sdk_2_05_01_00/examples/nortos/LP_MSPM0G3507/driverlib/empty*. Click on the *ticlang* folder (single-click, not double-click), then click on the "Open" button. Finally, click on the "Finish" button to import the project.

Click on the Explorer icon on the upper-left corner of CCS to see the imported project. Explorer is on the left pane of CCS and it shows the projects you have created.




The project files are stored in a *workspace* folder. On Windows, it would be in *C:\Users\<user_name>\workspace_ccstheia\empty_LP_MSPM0G3507_nortos_ticlang*; on macOS, it would be in */Users/<user_name>/workspace_ccstheia/empty_LP_MSPM0G3507_nortos_ticlang*. The default workspace folder is *workspace_ccstheia*. (You can create a new one and change to it as the new default. Create the folder you desire. Go to Explorer. On the CCS menu bar, click on <File>< Open…>. Go to the desired folder and click on the "Open" button.)

On Explorer, click on the project name. Press the <F2> function to rename the project to something related to this lab assignment. We will import this empty project again. If we do not do any renaming, CCS will refuse to import a project that already exists in Explorer. Expand the project tree and double-click on the main C program file *empty.c* to bring it to the text editor.



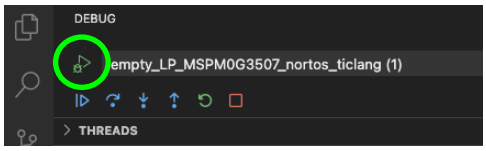

We are going to use the library function *printf*, so insert the following line in the *#include* section.

```
#include <stdio.h>
```

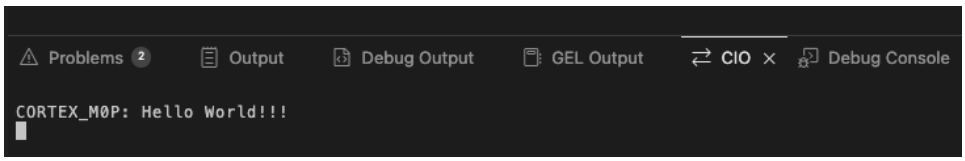
In the function *main()*, insert the following line before the *while* loop.

```
printf("Hello World!!!\n");
```

To build and download the program to the LaunchPad, click on the project name in Explorer. Press the <F5> function key. After everything is done, click on the triangle "continue" icon as shown below to run the program.




(Section 6. "Projects and Build" in the CCS user's guide provides details on the build process.) We should see the output displayed on the console (CIO) in the lower part of the CCS window.

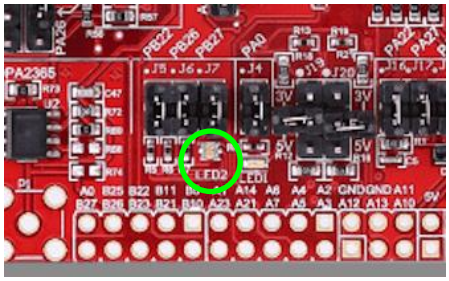



**Lab Report Submission**

1. Include a screenshot of the CIO showing the console tab and the outputs.
2. List the entire program in the appendix.

## 5. Exercise 3 Blink LED

In this exercise, we will import a small example project to blink an LED. Use the same method to import (and rename) the project in the previous exercise. On the CCS menu bar, click on <File>< Import Project(s)…>. If you use a Windows machine, you would go to *C:\ti\mspm0_sdk_2_05_01_00\examples\nortos\LP_MSPM0G3507\driverlib\gpio_toggle_output*. Click on the ticlang folder, then click on the "Select Folder" button. Then click on the "Finish" button to import the project. Build and execute the program. You should see LED2 on the board blinking with red and bluish white colors.



On the source file *gpio_toggle_output.c*, change the macro DELAY from 16000000 to 4000000 (1/4 of the original value). Rebuild and run the program again. You should see the blinking is much faster.

You have successfully used the DriverLib to build a small application!

Take a short video clip of the boarding blinking the LED. Place the video file where it can be accessed remotely.

**Lab Report Submission**

1. Include a picture showing the LED lit.
2. Include a link to the video clip. Do not submit the video file to Canvas. Make sure the video file can be opened by just clicking on the link.
3. List the entire program in the appendix.

## 6. Exercise 4 Access MCU Info

In this exercise, we will find out some device information about the MCU. You will use this exercise to refresh a bit of your C programming skills, if you have not used it for a while.

There is some specific information about the MCU that is stored in the device's FACTORY memory region, which is read-only. Each memory location is referred to as a *FACTORYREGION* register in the MCU technical reference (Section Factory Constants).

Create a new project, either by importing the example *empty* project or duplicating an existing one in Explorer. Be sure to re-name the project properly. We will create a program to read and print the information stored in the FACTORY region to the console. There are 32 data items. Note that each data item is stored as a 32-bit word.

You will find the starting address of the FACTORY region in the MCU technical reference for. Print the contents line by line. You are not allowed to use any library function, nor any predefined data structure in the DriverLib, to access the information. With the physical memory address you found, create a pointer to read different data items in the memory region. Use a character string array to store the acronyms (see Table 1-116) of the data items in your program. Note that only values are stored in the device's memory, not the acronym names. Print one data item on one line. Each line contains the acronym, the memory address, and the value. The addresses and values shall be displaced as 8-digit hexadecimal numbers.

Since we are going to use *printf*, make sure to insert the following line in the *#include* section of the main C file.

```
#include <stdio.h>
```

Be sure to properly declare the FACTORY region pointer so that it can be used to access a 32-bit word; use data type of specific bit width. Use a *for* or *while* loop to move from one data item to the next until you reach the end of the region.

### Lab Report Submission

1. List the code snippet that reads and prints all the information. Do not provide a screenshot to show the code; copy and paste the code as text.
2. List the outputs on the console. Do not take a screenshot; copy and paste the text outputs.
3. List the entire program in the appendix.

## 7. Submission Requirements

Write a formal report that contains at least what are required to be submitted in the exercises. Submit the report in PDF to Canvas by the deadline. Include program source code in your report. Do not use screenshots to show your codes. In the report body, if requested, list the relevant codes or screenshots only for the exercise you are discussing. Put the entire program listing (typically, the main C file) in the appendix. Use single-column format for the report.

In your report, show the console outputs (when requested) in text format; copy and paste them to your report. In general, do not use screenshot to show any text outputs or data; otherwise, points will be deducted.

When presenting a screenshot in the report, only present the relevant information. You may need to crop them from the initial screenshot. Please use proper scale so that the contents can be read easily.

Your report should have several sections. The first section is the information header that contains at least your name, course name and lab assignment number. Each exercise should have its own section where you discuss the results of the exercise. At the end, if requested, there should be the appendix that contains the complete source code for each exercise. The source code may be used to reproduce your reported results during grading. So, make sure they do produce the results you reported.

## 8. Grading Policy

The lab's grade is primarily determined by the report. If you miss the submission deadline for the report, you will get zero point for the lab assignment.

If your program (listed in the appendix) fails to compile, you will get zero point for the exercise.

If your program's outputs do not match what you reported, you will get zero point for the exercise.

Points may be deducted for incorrect statements, typos, non-readable texts on screenshots, lack of description on the graphs/pictures shown, etc. So, proofread your report before submission.

Pay attention to the required program behaviors described in the assignment. Points will be deducted for not producing the required outcomes.

## 9. Handling of LaunchPad

Use the following guidelines to physically handle the LaunchPad to reduce the chances of damaging it.

There is a dark bag holding the board. Do not throw away the bag. It is an anti-static bag. It would protect the board from static electricity. Keep the board inside the bag as much as possible; take it out only when you need to access it.

Do not touch the components on the board. Only hold the board by the edges.

Do not hold the board and walk across a room or bring it to another place. Always put it in the box or the anti-static bag first.